Дисциплина: Компьютерная графика
Выполнил: студент гр. 4, Попов Данила Денисович
Тема: Исследование цветовых моделей и их программная реализация

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

1. Цель и задачи работы

Цель работы: Изучить математический аппарат и физический смысл базовых цветовых пространств: аддитивной модели (RGB), субтрактивной модели (CMYK) и перцептивной модели (HSV). Разработать программное обеспечение (веб-приложение) для интерактивного пересчета цветовых координат между указанными моделями в режиме реального времени.

2. Теоретические основы и формулы преобразований

Цветовая модель представляет собой абстрактный математический инструмент для описания цветов с помощью числовых кортежей. В рамках данной работы рассматриваются три основные модели:

- **RGB (Red, Green, Blue)** — аддитивная модель, применяемая в мониторах и экранах. Цвет формируется за счет сложения лучей света. Диапазон каждой компоненты: [0, 255].
- **CMYK (Cyan, Magenta, Yellow, Key)** — субтрактивная модель для полиграфии, основанная на свойстве красок поглощать свет. Четвертый канал (K) отвечает за формирование глубокого черного цвета. Диапазон: [0, 100]%.
- **HSV (Hue, Saturation, Value)** — перцептивная (цилиндрическая) модель. Интуитивно понятна человеку: H (цветовой тон) задается углом [0, 360°], S (насыщенность) и V (яркость) — в диапазоне [0, 100]%.

В основе программной реализации лежит алгоритм, использующий RGB в качестве промежуточного «моста» для всех преобразований. Ниже приведены подробные математические алгоритмы.

1. Преобразование RGB → CMYK:
Нормализация координат: $R' = R/255, G' = G/255, B' = B/255$.
Вычисление компоненты черного цвета (Key):
$K = 1 - max(R', G', B')$.
Если $K = 1$, *цвет является чистым черным, и хроматические компоненты обнуляются:* $C = 0, M = 0, Y = 0$.
В противном случае вычисляются остальные каналы:
$C = (1 - R' - K) / (1 - K)$
$M = (1 - G' - K) / (1 - K)$
$Y = (1 - B' - K) / (1 - K)$
Итоговые значения (C, M, Y, K) умножаются на 100 для получения процентов.

2. Преобразование CMYK → RGB:
Нормализация из процентов в доли единицы: $C' = C/100, M' = M/100, Y' = Y/100, K' = K/100$.
Восстановление цветовых лучей с учетом поглощения (вычитания) света компонентами и общей черной краской:

3. Преобразование RGB → HSV:
Обозначим экстремумы: Max = max(R′, G′, B′), Min = min(R′, G′, B′), хроматический размах Δ = Max − Min.
Цветовой тон (H) вычисляется по секторам круга:

- *H = 0°, если Δ = 0 (ахроматический цвет)*
- *H = 60° × (((G′ − B′) / Δ) mod 6), если Max = R′*
- *H = 60° × (((B′ − R′) / Δ) + 2), если Max = G′*
- *H = 60° × (((R′ − G′) / Δ) + 4), если Max = B′*

Насыщенность (S): если Max = 0, то S = 0; иначе S = Δ / Max.
Яркость (V): V = Max.
Итоговые S и V умножаются на 100%.

4. Преобразование HSV → RGB:
Нормализация процентов: S′ = S/100, V′ = V/100. Угол H не меняется.
Вычисляются вспомогательные хроматические параметры:
C (Chroma) = V′ × S′
X = C × (1 − |(H / 60°) mod 2 − 1|)
Модуль согласования яркости: m = V′ − C.
Промежуточные координаты (R_1, G_1, B_1) определяются в зависимости от сектора H:

- *0° ≤ H < 60°: (C, X, 0)*
- *60° ≤ H < 120°: (X, C, 0)*
- *120° ≤ H < 180°: (0, C, X)*
- *180° ≤ H < 240°: (0, X, C)*
- *240° ≤ H < 300°: (X, 0, C)*
- *300° ≤ H < 360°: (C, 0, X)*

Окончательные значения каналов получают добавлением яркости m и масштабированием:
$R = (R_1 + m) \times 255$
$G = (G_1 + m) \times 255$
$B = (B_1 + m) \times 255$.

3. Архитектура программного решения

Проект реализован по паттерну клиент-серверного микросервиса, что позволяет отделить интерфейсную часть от вычислительной логики:

- **Backend (Python / Flask):** Сервер принимает JSON-запросы, маршрутизирует их и выполняет математические преобразования цветовых моделей с использованием чисел с плавающей точкой (для сохранения точности расчетов).
- **Frontend (HTML / CSS / JS):** Клиентская часть отвечает за реактивность интерфейса. Обновление значений в любом поле ввода (или сдвиг ползунка) инициирует асинхронный fetch-запрос к API сервера.

4. Результаты работы

Ниже представлен интерфейс разработанного веб-приложения, демонстрирующий мгновенную синхронизацию моделей CMYK, RGB и HSV при выборе цвета. Приложение корректно обрабатывает ввод через текстовые поля, ползунки (range) и системную палитру.

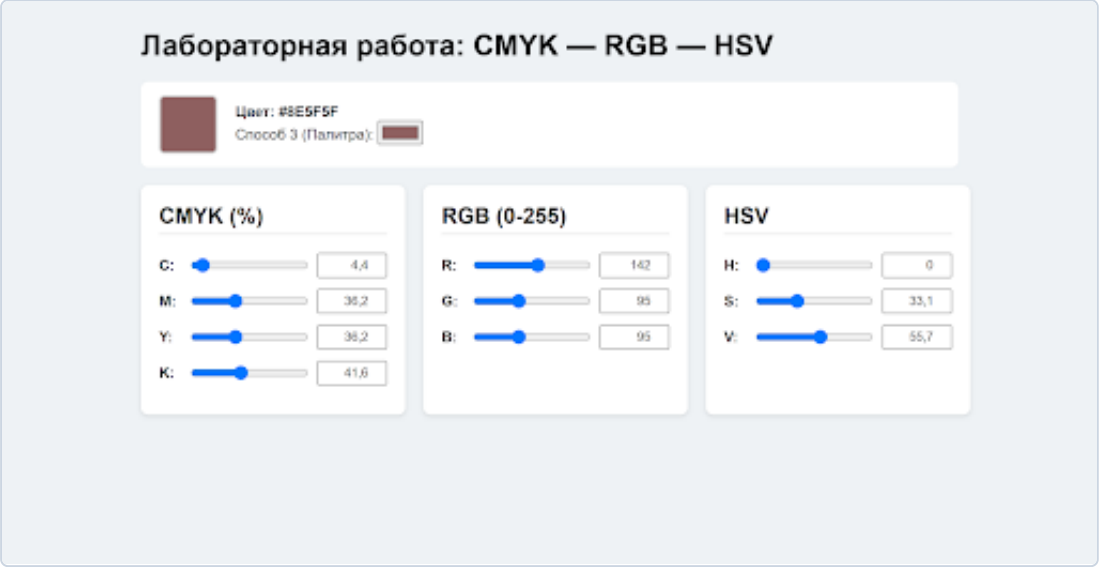


Рис 1. Интерактивный пересчет цветовых моделей в реальном времени

5. Выводы

В ходе выполнения лабораторной работы были исследованы математические принципы базовых цветовых моделей и успешно реализован алгоритм их взаимного преобразования. Тестирование программы позволило сделать следующие выводы:

1. **Специфика вырождения координат:** При преобразованиях обнаружены точки необратимости. Например, в модели HSV при нулевой насыщенности (ахроматические цвета, оттенки серого) параметр цветового тона (H) математически не определен (утрачивает смысл) и обычно приравнивается к нулю.
2. **Вариативность черного в CMYK:** Идеализированный алгоритм всегда выделяет максимум черного в канал K (Key). Однако на практике в полиграфии генерация черного может варьироваться (GCR/UCR) для достижения одной и той же визуальной плотности цвета разными комбинациями красок.
3. **Вычислительная точность:** Для исключения эффекта накапливающейся ошибки при перетаскивании ползунков, все промежуточные пересчеты на сервере (внутри Python) ведутся с использованием вещественных чисел (float). Округление применяется исключительно на этапе рендеринга значений в пользовательском интерфейсе.